SATQUERY AI

Role 1 - AI Agent Developer

Controller, orchestration, memory, reliability and evidence

PROJECT FOCUS	An agentic vision-language assistant that turns natural-language queries over remote-sensing imagery into routed, auditable specialist workflows.
ROLE 1 OUTCOME	Build the decision-making layer that interprets a query, validates available inputs, selects the correct specialist workflow, executes it through structured tools, handles uncertainty and failure, preserves context across turns, and returns an auditable result.

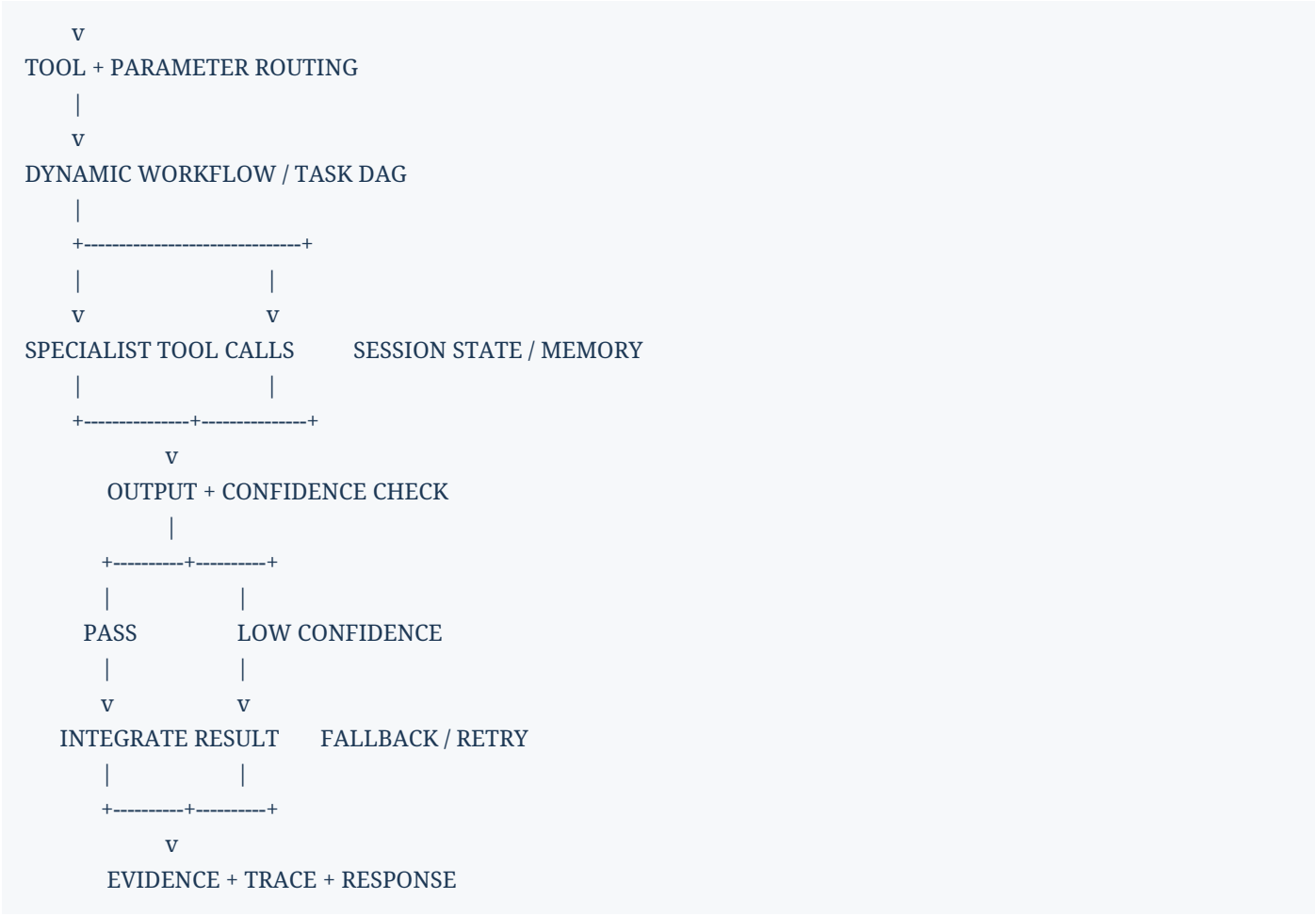
1. What Role 1 owns

Role 1 is the control layer. It turns unstructured user intent and available image context into an executable workflow, then combines specialist outputs into one grounded response.

- **Query intent classification:** Classify natural-language queries into target workflows such as SINGLE_VQA, REGION_GROUNDING, BI_TEMPORAL_CHANGE_DETECTION and CROSS_MODAL_OPTICAL_SAR.
- **Input compatibility validation:** Check whether the supplied images and metadata satisfy the selected workflow. For bi-temporal analysis, this includes validating that the pair is usable together and has compatible spatial context.
- **Dynamic tool orchestration:** Maintain a registry of specialist tools and route requests through a directed workflow. Tools are selected by contract rather than by model-specific implementation details.
- **Parameter assignment:** Translate user intent and available context into structured tool parameters, including text targets, image references, time context and spatial information.
- **Output integration and confidence:** Combine textual outputs with bounding boxes, masks or other spatial evidence and calculate an overall confidence view.
- **Auditable execution trace:** Record selected tasks, model/tool IDs, parameters, timings, status and output references so the final answer can be reconstructed.

2. Role 1 execution flow

```
USER QUERY + AVAILABLE IMAGE CONTEXT
      |
      v
INTENT CLASSIFICATION
      |
      v
INPUT / CONTEXT VALIDATION
      |
```



3. Specialist tool registry

Role 1 should expose a stable interface for each core workflow. The router decides which contract to invoke; the specialist implementation stays behind that contract.

Tool	Inputs	Expected output
single_image_vqa	image_path, query	Text answer / visual interpretation
region_grounding	image_path, target_object	Bounding boxes / grounded regions
bitemporal_change_analysis	img_t1, img_t2, query	Change mask + text
cross_modal_fusion	optical_img, sar_img, query	Joint analysis text

4. Stateful multi-turn memory

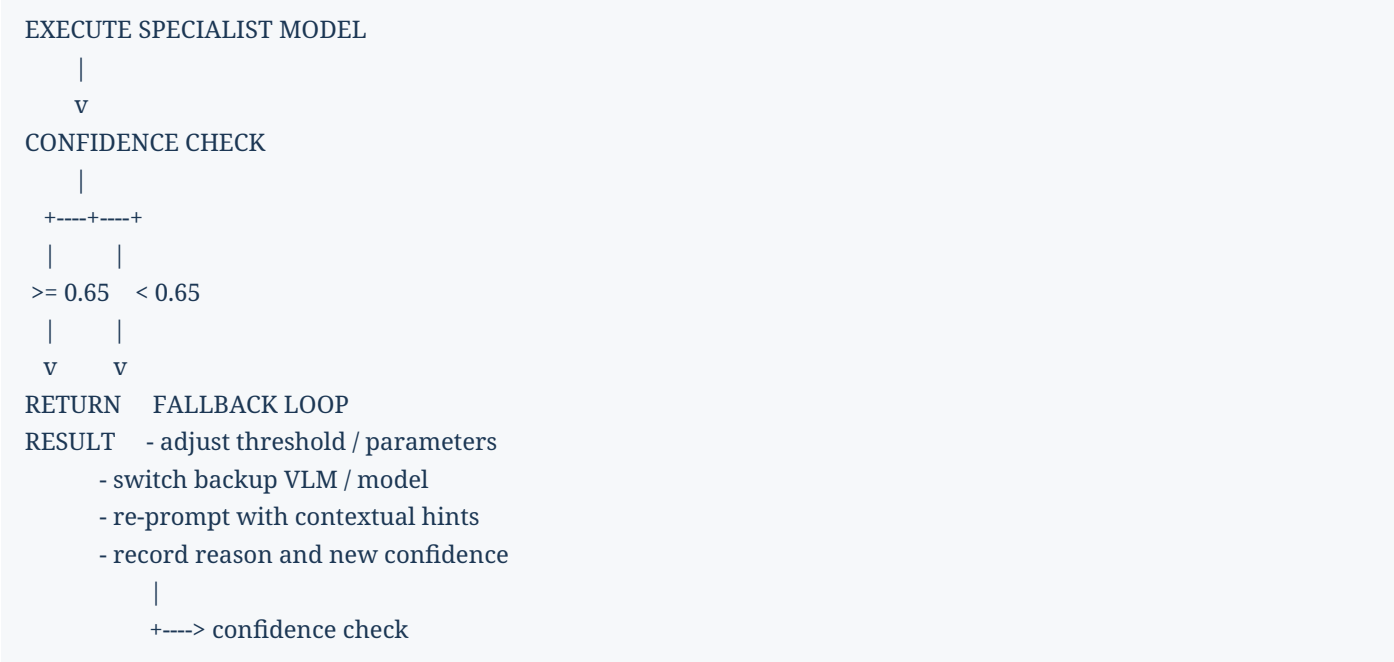
The agent must preserve the active visual context across follow-up turns. Example: a user can first ask for a land-cover description and then ask the agent to highlight water bodies in the same image without re-uploading it.

- **State that persists:** session_id, request history, active_image_ids and relevant spatial context.
- **Transient execution state:** current tool, current confidence, intermediate mask references and other information needed only while a run is active.

- **Implementation direction:** Use LangGraph Checkpointers such as MemorySaver or a Postgres-backed checkpointer, linked to a session_id. Store active image metadata in the agent state rather than repeatedly asking the user for context.
- **Design rule:** Keep stable identifiers separate from transient state. Do not put unnecessary binary image data into the graph state.

5. Confidence, fallback and self-correction

A low-confidence model output should be treated as a routing event, not as a terminal failure. The controller evaluates the result and decides whether to accept, retry, or switch to a backup path.



- **Bounded retries:** Define a maximum retry count and a clear terminal response when every candidate path fails.
- **Failure taxonomy:** Classify failures as input validation, metadata/context issue, model timeout, low confidence, dependency failure or unexpected exception.
- **Observable decisions:** Record why a retry happened, what parameter changed, which backup path was tried and the resulting confidence.
- **Deterministic demo path:** Maintain a stable mock/fallback path so the agent workflow can still be demonstrated when a large model is unavailable or slow.

6. Auditable execution trace

The trace is part of the agent output, not an afterthought. It should make the system explainable at the workflow level: what was selected, what ran, how long it took, what confidence it produced and what evidence was returned.

Trace field	Purpose
request_id	Reconstruct one execution end to end.
session_id	Tie multiple turns to the same conversation state.

selected_workflow	Show why a specialist path was chosen.
tool/model name	Identify the actual capability invoked.
input IDs / parameters	Show what the tool received.
duration + status	Measure execution and locate failures.
confidence	Make acceptance/fallback decisions inspectable.
output references	Point to masks, evidence and generated reports.
fallback reason	Explain self-correction behaviour.

7. Role 1 technology stack

Area	Recommended technology	Role 1 purpose
Agent framework	LangGraph / custom async Python agent	Stateful DAG orchestration and tool calling.
Tool contracts	Pydantic	Strict input/output schemas and structured validation.
LLM/VLM routing	OpenAI / Gemini API / local vLLM instance	Intent parsing and tool-parameter extraction.
State persistence	LangGraph Checkpointers	Multi-turn session persistence.
Trace/reporting	JSON Schema + ReportLab / WeasyPrint	Structured trace validation and downloadable reports.

8. Learning and implementation roadmap

Phase 1 - Core fundamentals (Days 1-7)

Agent architecture and tool orchestration

What to learn: Learn state machines, tool registration, JSON schema generation, structured outputs and fallback loops.

Hands-on: Build a small Python agent that selects between a Weather Tool and Math Tool, executes via Pydantic and writes an auditable trace.

Phase 2 - Intermediate integration (Days 8-15)

Agent tool registry and multimodal DAG routing

What to learn: Design contracts for the four core remote-sensing workflows and implement the model router.

Hands-on: Use mock model functions that return dummy spatial coordinates and text summaries to prove end-to-end routing.

Phase 3 - Advanced agentic features (Days 16-25)

Auditable trace and confidence scoring

What to learn: Learn execution-log structure, confidence aggregation and report generation.

Hands-on: Generate JSON and HTML/PDF traces containing timing, model name, confidence and input parameters.

Captain-aligned additions

Multi-turn memory

What to learn: Learn LangGraph Checkpointers, session identifiers and active visual-context state.

Hands-on: Demonstrate a second user turn that reuses an image context without re-upload.

Fallback and self-correction

What to learn: Learn conditional branching in LangGraph based on confidence evaluation.

Hands-on: Trigger a low-confidence result and show bounded retry / backup-model behaviour.

Structured tool contracts

What to learn: Master pydantic.BaseModel for tool schemas accepting both spatial and text inputs.

Hands-on: Validate tool inputs and outputs before connecting real specialist models.

9. Engineering controls for Role 1

Control	Why it matters
Contract tests	Validate each tool wrapper against the same request/response schema before plugging in a real model.
Golden demo case	Keep one known image/query pair whose route, trace shape and output type are stable.
Failure taxonomy	Keep validation failures, low confidence, timeouts and dependency failures distinguishable.
Request identifiers	Carry session_id + request_id + image_id through the execution so a run is reconstructable.
Resource limits	Set practical limits for upload size, processing time, concurrency and raster memory assumptions exposed to the agent.
Fallback determinism	Define maximum retries and a clear terminal response.
Schema versioning	Version contracts so new fields do not silently break older components.
Demo observability	Maintain one simple trace view or downloadable execution record that proves what the agent did.

10. Recommended Role 1 project structure

```
satquery_ai/
|-- agent/
|   |-- agent_engine.py      # LangGraph state graph
|   |-- tools/               # VQA, Grounding, Change, Fusion wrappers
|   |-- router.py            # Intent classifier / route selection
|   |-- state.py             # Session + visual context state
|   |-- validators.py        # Input and contract validation
|   |-- confidence.py        # Confidence aggregation / gates
|   |-- fallback.py          # Retry + backup routing
```

```
|   |-- tracer.py          # Auditable trace generator
|   |-- schemas.py           # Pydantic contracts
|   `-- reports.py         # JSON / HTML / PDF evidence output
`-- tests/
    |-- test_contracts.py
    |-- test_routing.py
    |-- test_memory.py
    |-- test_fallback.py
    `-- test_trace.py
```

11. Development support with main_A_agent

main_A_agent is an implementation aid. The source of truth remains the team repository, agreed interfaces, tests and architecture decisions.

- **Architecture and schemas:** Use main_A_agent to draft Pydantic tool contracts, agent state models and trace schemas.
- **Agent controller assembly:** Use it to accelerate LangGraph state-machine code for intent classification, validation, routing and fallback.
- **Debugging:** Use it to inspect stack traces and reason through state errors, async issues, schema mismatches or routing bugs.
- **Trace and evidence output:** Use it to draft JSON/PDF trace generation and visual evidence formatting.
- **Test generation:** Use it to produce contract tests, routing cases, memory tests and fallback scenarios.

12. Immediate execution order

Step	Deliverable
Define contracts	Freeze Pydantic schemas for session state, tool inputs/outputs, confidence, errors and trace records.
Build the smallest vertical slice	One image + one query -> validate -> classify -> call one mock tool -> return structured text + trace.
Add statefulness	Persist session_id and active visual context, then prove a follow-up turn can reuse the same image.
Add routing breadth	Register the four core specialist workflows and exercise the DAG with mock models.
Add reliability	Introduce confidence gates, bounded fallback loops and explicit failure classes.
Add evidence	Return spatial evidence, confidence and downloadable trace output as one structured result.
Swap mocks for real models	Integrate specialist implementations only after the contracts and tests are stable.

13. Definition of done

Stage	Definition
D0 - Contracted	Schemas exist for tools, state, confidence, errors and traces.
D1 - Vertical slice	One image + one query completes end to end using mock

	inference.
D2 - Spatially aware	Relevant image context and spatial references are preserved through the workflow.
D3 - Stateful	A follow-up user turn reuses the same image context without re-upload.
D4 - Resilient	Low-confidence results trigger a bounded fallback/retry path and remain observable.
D5 - Tool-complete	The four core workflows are registered behind stable contracts.
D6 - Auditable	Trace contains route, tools, parameters, timings, confidence, status and outputs.
D7 - Demo ready	Real model or deterministic fallback path and trace generation work together.

Implementation note: keep the first demo narrow, make decisions observable, and expand specialist coverage only after the controller contracts and failure behaviour are stable.

14. Role 1 verification checklist

Area	Verification
Intent routing	Each supported query type reaches the intended workflow with deterministic route selection.
Input validation	Missing or incompatible image context is detected before specialist execution.
Tool contracts	Every registered tool passes request/response contract tests.
Session memory	A follow-up turn can reuse the active visual context without re-upload.
Confidence gate	A confidence score is evaluated before the result is accepted.
Fallback	Low-confidence or failed executions follow a bounded retry/backup path.
Trace	Every run records IDs, route, parameters, timing, status, confidence and outputs.
Evidence	The final result can point to the relevant spatial evidence or output reference.
Demo mode	A known image/query pair can complete through a deterministic mock/fallback path.
Tests	Routing, memory, fallback and trace behaviour are covered by repeatable tests.